

# **SIGN2SPEAK : WHERE SILENCE FINDS IT'S VOICE**

## **A PROJECT REPORT**

*Submitted by*

B RUSHINDRA                      (24MIM10145)  
B V AAKANKSH                      (24MIM10059)  
G ROHITH REDDY                      (24MIM10100)  
D CHANDRA SHEKHAR(24MIM10066)

*in partial fulfillment for the award of the degree*

*of*

**INTEGRATED MASTER OF TECHNOLOGY**

*in*

**PROGRAMME OF STUDY**



**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

**VIT BHOPAL UNIVERSITY**

**KOTHRIKALAN, SEHORE**

**MADHYA PRADESH-466114**

**APRIL, 2026**

**VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE,  
MADHYA PRADESH – 466114**

**CERTIFICATE**

This is to certify that the thesis titled **SIGN2SPEAK - Where Silence Finds its Voice** submitted by **B Rushindra (24MIM10145), D Chandra Shekar (24MIM10066), B V Aakanksh (24MIM10059), Rohith Reddy (24MIM10100)** to VIT Bhopal University, in partial fulfillment of the requirement for the award of the degree of **Integrated Master of Technology in Artificial Intelligence** is a bona-fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this thesis have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

**Supervisor**

Signature: \_\_\_\_\_

Name: Dr. Nilamadhab Mishra

Designation: Assistant Professor

Date: \_\_\_\_\_

**Program Chair**

Signature: \_\_\_\_\_

Name: Dr.S Devaraju

Designation: Assistant Professor

Date: \_\_\_\_\_

**Reviewer 1**

Signature: \_\_\_\_\_

Name: Dr. Paras Jain

Designation: Senior Associate Professor

Date: \_\_\_\_\_

**Reviewer 2**

Signature: \_\_\_\_\_

Name: Dr. Trapti Sharma

Designation: Assistant Professor

Date: \_\_\_\_\_

A viva-voce was conducted on .....

## ACKNOWLEDGEMENT

I am deeply indebted to **Dr. G. Viswanathan**, Chancellor of VIT Bhopal University, and **Dr. Sathish Kumar Modh**, Vice Chancellor of VIT Bhopal university, **Dr. T.B. Sridharan**, Pro Vice-Chancellor of VIT Bhopal University, for their unwavering support and guidance. Their leadership and vision have been a constant source of inspiration throughout my academic journey.

Express my profound gratitude to **Dr. Pon Harshavardhan**. Dean of SCAl, for his encouragement and invaluable support at every stage, which significantly contributed to the successful completion of my project and degree.

My heartfelt thanks go to **Dr. S. Devaraju**, Program Chair of Integrated M.Tech-ArtificialIntelligence, and **Dr. Venkat Prasad Padhy**, Division Head of the AI Division, for their timely assistance, guidance, and invaluable insights during the course of my research.

A special note of appreciation reserved for my supervisor, **Dr. Nilamadhab Mishra**, whose constant support, patience, and expertise were instrumental in completing this project. His guidance, encouragement, and clarity in addressing my queries made this journey both meaningful and enriching.

I would like to express my sincere gratitude to **Dr. Paras Jain** and **Dr. Trapti Sharma** for their valuable feedback and insightful reviews. Lastly, I extend my deepest gratitude to my parents and friends, whose unwavering encouragement and support have been my driving force throughout this research endeavor. Their belief in me has been a source of strength and motivation

## LIST OF ABBREVIATIONS

Based on the text in the document, here are the abbreviations and their corresponding full forms:

| Abbreviation | Full Form/Meaning             |
|--------------|-------------------------------|
| AI           | Artificial Intelligence       |
| AR           | Augmented Reality             |
| CNN          | Convolutional Neural Networks |
| CSV          | Comma-Separated Values        |
| DL           | Deep Learning                 |
| DPT          | Dense Prediction Transformer  |
| FR           | Functional Requirements       |
| GUI          | Graphical User Interface      |
| I/O          | Input/Output                  |
| ISL          | Indian Sign Language          |
| NFR          | Non-Functional Requirements   |
| OBJ          | Object (file format)          |
| OOD          | Object-Oriented Design        |
| OS           | Operating System              |
| SfM          | Structure from Motion         |
| UML          | Unified Modeling Language     |
| UX           | User Experience               |
| VR           | Virtual Reality               |

## LIST OF FIGURES

| <b>FIGURE NO.</b> | <b>TITLE</b>                                                                   | <b>PAGE NO.</b> |
|-------------------|--------------------------------------------------------------------------------|-----------------|
| <b>4.1</b>        | Sign2Speak system pipeline converting hand gestures into real-time text output | <b>12</b>       |
| <b>5.1</b>        | Demonstration of Hand Landmark detection using Mediapipe                       | <b>19</b>       |
| <b>6.1</b>        | Survey on how many know sign language                                          | <b>25</b>       |
| <b>6.2</b>        | survey on How Difficult it becomes to communicate with a deaf person.          | <b>25</b>       |
| <b>6.3</b>        | What problems Might Occur                                                      | <b>25</b>       |
| <b>6.4</b>        | Would AI be useful in breaking the communication barrier                       | <b>26</b>       |
| <b>6.5</b>        | Where would this system be most useful                                         | <b>26</b>       |

## **ABSTRACT**

The Vision Morph AI project establishes a fast, robust, monocular 2D-to-3D conversion pipeline to generate high-fidelity 3D models from single 2D images. This system addresses the inherent challenge of lacking explicit depth information in 2D media and is designed to democratize 3D asset creation for applications in Virtual Reality (VR), Augmented Reality (AR), and gaming through an accessible Graphical User Interface (GUI). The core objective is to surpass the limitations and dependencies of traditional multi-view reconstruction methods by leveraging advanced Deep Learning (DL) architectures. The methodology employs a dual-approach architecture orchestrated by the ImageTo3DConverter class, contrasting a simple Heightmap Converter (grayscale intensity to Z-elevation) with a primary DL-based method.

The DL approach utilizes the Dense Prediction Transformer (DPT) model to accurately infer relative scene depth. Both resulting depth representations are processed by the Mesh Processor for surface triangulation, optional post-processing (smoothing and decimation), and final export to the standard OBJ format. Testing confirmed the feasibility of the high-quality conversion, with the DPT method yielding geometrically superior and more accurate models compared to the heightmap baseline. The system's success validates its effectiveness as a powerful tool for rapid 3D prototyping and asset creation

## LIST OF TABLES

| Table No | Title                | Page No |
|----------|----------------------|---------|
| 4.1      | Implementation Table | 14      |

## TABLE OF CONTENT

| CHAPTE<br>R NO. | TITLE                                                     | PAGE<br>NO. |
|-----------------|-----------------------------------------------------------|-------------|
|                 | List of Abbreviation                                      | iv          |
|                 | List of Figures                                           | v           |
|                 | Abstract                                                  | vi          |
|                 | List of Tables                                            | vii         |
| 1.              | <b>CHAPTER-1:PROJECT DESCRIPTION AND OUTLINE</b>          |             |
|                 | 1.1 Introduction                                          | 1           |
|                 | 1.2 Motivation to do this Project                         | 1           |
|                 | 1.3 Problem Statement and Scope Definition                | 2           |
|                 | 1.4 Objectives and Deliverables                           | 2           |
|                 | 1.5 Organization of the Project Report                    | 3           |
|                 | 1.6 Summary                                               | 3           |
| 2.              | <b>CHAPTER 2 – RELATED WORK INVESTIGATION</b>             |             |
|                 | 2.1 Introduction to Sign Language Recognition Systems     | 4           |
|                 | 2.2 Classical Sensor-Based and Vision-Based Techniques    | 4           |
|                 | 2.2.1 Sensor Based Glove Systems                          | 4           |
|                 | 2.2.2 Traditional Computer Vision Techniques              | 5           |
|                 | 2.2.3 Limitations of Traditional Method                   | 5           |
|                 | 2.3Deep Learning Approaches for Sign Language Recognition | 5           |
|                 | 2.3.1 Evolution of Recognition Architectures              | 6           |
|                 | 2.3.2 MediaPipe-Based Hand Landmark Detection             | 6           |
|                 | 2.3.3 Deep Learning Models using Tensorflow               | 7           |
|                 | 2.3.4 Real -Time Gesture Recognition Systems              | 7           |
| 3.              | <b>CHAPTER 3 – REQUIREMENT ARTIFACTS</b>                  |             |
|                 | 3.1 Introduction to System Requirements                   | 8           |
|                 | 3.2 Functional Requirements                               | 8           |
|                 | 3.2.1 Input?Output Management                             | 8           |
|                 | 3.2.2 Hand Landmark Detection using MediaPipe             | 9           |



|    |                                                           |    |
|----|-----------------------------------------------------------|----|
|    | 3.2.3 Gesture Classification using Tensorflow Model       | 9  |
|    | 3.2.4 Real-Time Prediction and Visualization              | 9  |
|    | 3.2.5 Data Processing and Generation                      | 10 |
|    | 3.3 Non-Functional Requirements (NFR)                     | 10 |
|    | 3.3.1 Performance                                         | 10 |
|    | 3.3.2 Usability and User Interface                        | 10 |
|    | 3.3.3 Implementation and Maintenance                      | 11 |
|    | 3.4 Requirement Traceability Matrix                       | 11 |
| 4. | <b>CHAPTER 4 – DESIGN METHODOLOGY AND ITS NOVELTY</b>     |    |
|    | 4.1 Introduction                                          | 12 |
|    | 4.2 System Architecture                                   | 13 |
|    | 4.2.1 Architectural Breakdown                             | 13 |
|    | 4.3 Implementation Table                                  | 14 |
|    | 4.4 Object-Oriented Design(UML Class Diagram)             | 15 |
|    | 4.4.1 Core Classes and Responsibilities                   | 15 |
|    | 4.5 Novelty and Unique Contributions                      | 16 |
| 5. | <b>CHAPTER 5 – TECHNICAL IMPLEMENTATIONS AND ANALYSIS</b> |    |
|    | 5.1 Introduction                                          | 18 |
|    | 5.2 Implementation Stack and Dependencies                 | 18 |
|    | 5.3 Core Module Implementation                            | 19 |
|    | 5.3.1 Hand Landmark Detection using MediaPipe             | 19 |
|    | 5.3.2 Data Preprocessing and Normalization                | 20 |
|    | 5.3.3 Gesture Classification using TensorFlow Model       | 20 |
|    | 5.3.4 Real Time Processing and Optimization               | 20 |
|    | 5.3.5 Web Application Integration ( Flask Backend )       | 21 |
|    | 5.4 System Orchestration and Processing Flow              | 21 |
|    | 5.5 Validation and Performance Analysis Plan              | 21 |
|    | 5.5.1 Validation Testing (Functional Compliance)          | 21 |
|    | 5.5.2 Quantitative Benchmarking                           | 22 |

|    |                                                      |    |
|----|------------------------------------------------------|----|
| 6. | <b>CHAPTER 6 – PROJECT OUTCOME AND APPLICABILITY</b> |    |
|    | 6.1 Outline                                          | 23 |
|    | 6.2 Final System Demonstration and Key Results       | 23 |
|    | 6.3 Case Studies                                     | 24 |
|    | 6.3.1 Case Study 1: Educational Assistance           | 24 |
|    | 6.3.2 Case Study 2: Real-Time Communication Support  | 24 |
|    | 6.3.3 Case Study 3: Public and Service Application   | 24 |
|    | 6.4 User Survey and Feedback Analysis                | 24 |
|    | 6.5 Project Applicability on Real-world Applications | 26 |
|    | 6.6 Inference                                        | 27 |
| 7. | <b>CHAPTER 7 – CONCLUSIONS AND RECOMMENDATION</b>    |    |
|    | 7.1 Summary of Work Accomplished                     | 28 |
|    | 7.2 Limitation/Constraints of the System             | 28 |
|    | 7.3 Future Enhancements and Research Directions      | 29 |
|    | 7.4 Inference                                        | 29 |
|    | Appendix A                                           | 31 |
|    | Appendix B                                           | 33 |
|    | References                                           | 35 |

# CHAPTER 1

## PROJECT DESCRIPTION AND OUTLINE

### 1.1 Introduction

Communication is an essential part of human life, enabling individuals to express thoughts, ideas, and emotions effectively. However, for people with hearing and speech impairments, communication becomes challenging as they primarily rely on sign language, which is not widely understood by the general population. This creates a significant communication gap between differently-abled individuals and society. The project titled **“SIGN2SPEAK – Where Silence Finds Its Voice”** is designed to address this issue by developing an intelligent system capable of recognizing sign language gestures and converting them into readable text in real time. The system utilizes advanced technologies such as TensorFlow, MediaPipe, Convolutional Neural Networks (CNN), and computer vision techniques to accurately detect and classify hand gestures. By integrating machine learning with a user-friendly interface, the project aims to provide an accessible and efficient communication tool that can be used in everyday life.

### 1.2 Motivation to Do This Project

The primary motivation behind this project arises from the need to create an inclusive society where communication barriers are minimized. Hearing and speech-impaired individuals often face difficulties in interacting with others due to the lack of understanding of sign language among the general population. Traditional solutions such as human interpreters are not always available and can be costly, while hardware-based systems are often complex and inconvenient. This project is motivated by the desire to leverage these technologies to build a system that can interpret sign language in real time using just a webcam. The goal is to empower individuals with disabilities by providing them with a tool that enhances their ability to communicate independently and confidently.

### **1.3 Problem Statement and Scope Definition**

The major problem addressed in this project is the communication gap between hearing-impaired individuals and people who do not understand sign language. Existing systems for sign language recognition either require specialized hardware, are limited in functionality, or lack real-time performance. Therefore, there is a need for a system that is accurate, real-time, cost-effective, and easy to use. The scope of this project includes the development of a real-time sign language recognition system that can detect and classify Indian Sign Language (ISL) gestures, specifically alphabets (A–Z) and numbers (1–9). The system uses a webcam for capturing input and processes the gestures using machine learning models. While the current implementation focuses on static gestures, the scope can be extended in the future to include dynamic gestures, word formation, and speech output for enhanced usability.

### **1.4 Objectives and Deliverables**

The main objective of this project is to design and implement a real-time sign language recognition system that can accurately identify hand gestures and convert them into text. This involves several sub-objectives such as capturing live video input, detecting hand landmarks using MediaPipe, preprocessing the extracted data, and classifying gestures using a trained CNN model built with TensorFlow. Another objective is to develop an interactive and user-friendly interface that displays predictions in real time. The deliverables of this project include a trained deep learning model capable of recognizing ISL gestures, a dataset of processed hand landmarks, a Flask-based web application for real-time interaction, and a complete project report documenting the methodology and results. The system is expected to achieve high accuracy and provide smooth real-time performance.

### **1.5 Organization of Project Report**

This project report is organized into several chapters to provide a structured understanding of the work carried out. Chapter 1 presents the project description and outline, including the introduction, motivation, objectives, and scope of the project.

Chapter 2 discusses the related work and existing systems in the field of sign language recognition. Chapter 3 outlines the requirement artifacts, including hardware and software requirements. Chapter 4 explains the design methodology and highlights the novelty of the proposed system. Chapter 5 provides a detailed explanation of the technical implementation, including data processing, model training, and system integration. Chapter 6 describes the project outcomes and its practical applications. Finally, Chapter 7 concludes the report with a summary of the work and suggestions for future improvements.

## **1.6 Summary**

In summary, the **Sign2Speak** project presents an innovative solution to the communication challenges faced by hearing and speech-impaired individuals. By combining computer vision, machine learning, and web technologies, the system enables real-time recognition of sign language gestures and converts them into readable text. The project not only demonstrates the practical application of artificial intelligence but also contributes towards building a more inclusive and accessible society. The successful implementation of this system highlights its potential for further development and real-world deployment in various domains such as education, healthcare, and public services

## **CHAPTER 2**

### **RELATED WORK INVESTIGATION**

#### **2.1 Introduction to Sign Language Recognition Systems**

The field of sign language recognition focuses on developing computational systems that can interpret hand gestures and convert them into meaningful text or speech. This domain combines concepts from computer vision, machine learning, and human-computer interaction to bridge the communication gap between hearing-impaired individuals and the general population. The investigation into related work is essential for understanding the evolution of gesture recognition systems, identifying the limitations of existing approaches, and justifying the selection of modern deep learning techniques in the Sign2Speak project. The literature in this domain can be broadly categorized into traditional sensor-based methods, vision-based classical approaches, and modern deep learning-based systems.

#### **2.2 Classical Sensor-Based and Vision-Based Techniques**

Before the widespread use of deep learning, sign language recognition systems relied heavily on hardware sensors and classical image processing techniques. These methods aimed to capture hand movements and gestures using external devices or handcrafted feature extraction algorithms.

##### **2.2.1 Sensor-Based Glove Systems**

Sensor-based systems use specialized gloves embedded with sensors such as accelerometers, flex sensors, and gyroscopes to capture hand movements and finger bending. These systems directly measure physical parameters of hand gestures and convert them into digital signals for classification. While such systems can achieve high accuracy due to precise data collection, they have several drawbacks. They are expensive, inconvenient to wear, and limit natural interaction. Additionally, they require users to

carry external hardware, making them unsuitable for widespread adoption in real-world scenarios.

### **2.2.2 Traditional Computer Vision Techniques**

Traditional vision-based approaches use cameras to capture hand gestures and apply image processing techniques for recognition. These methods typically involve steps such as image segmentation, edge detection, contour extraction, and feature engineering. Techniques like skin color detection and background subtraction are used to isolate the hand region, followed by feature extraction methods such as Histogram of Oriented Gradients (HOG) or Scale-Invariant Feature Transform (SIFT).

Although these methods reduce the need for external hardware, they are highly sensitive to lighting conditions, background noise, and variations in hand orientation. Moreover, handcrafted features often fail to generalize well across different users and environments, resulting in reduced accuracy and robustness.

### **2.2.3 Limitations of Traditional Methods**

While sensor-based and classical vision-based techniques laid the foundation for gesture recognition, they suffer from several limitations. Sensor-based systems are costly and impractical, while traditional computer vision methods lack robustness and scalability. Both approaches struggle with real-time performance and adaptability to different conditions. These challenges highlight the need for more advanced, flexible, and accurate solutions, leading to the adoption of deep learning-based approaches.

## **2.3 Deep Learning Approaches for Sign Language Recognition**

The introduction of deep learning has significantly transformed the field of sign language recognition. These methods eliminate the need for manual feature extraction by automatically learning patterns directly from data. Deep learning models treat gesture recognition as a classification problem, where input images or keypoint data are mapped to corresponding labels.

### **2.3.1 Evolution of Recognition Architectures (CNN to Keypoint-Based Models)**

Early deep learning approaches for sign language recognition were based on Convolutional Neural Networks (CNNs), which are highly effective for image classification tasks. CNNs learn spatial features such as edges, shapes, and textures, enabling them to recognize hand gestures from images. However, CNN-based models often require large datasets and can be computationally expensive.

To overcome these challenges, newer approaches incorporate keypoint-based models, where instead of processing raw images, the system extracts important hand landmarks. This reduces computational complexity and improves efficiency. Libraries like MediaPipe have made it possible to detect 21 hand landmarks in real time, providing a structured representation of hand gestures that can be easily processed by machine learning models.

### **2.3.2 MediaPipe-Based Hand Landmark Detection**

MediaPipe is a powerful framework developed for real-time hand tracking and gesture recognition. It detects 21 key points on the hand, including fingertips, joints, and the wrist, providing precise spatial information about hand movements. These landmarks are used to represent gestures in a compact and normalized form.

In the Sign2Speak project, MediaPipe plays a crucial role in feature extraction. Instead of relying on raw pixel data, the system processes these landmarks, making it more robust to variations in lighting, background, and hand orientation. This approach significantly improves performance and reduces computational requirements, making it suitable for real-time applications.

### **2.3.3 Deep Learning Models using TensorFlow**

TensorFlow is widely used for building and training deep learning models for gesture recognition. In this project, a neural network model is trained using processed landmark data to classify gestures into alphabets and numbers. Compared to traditional methods, deep learning models provide higher accuracy and better generalization.



The use of neural networks allows the system to learn complex patterns in hand gestures and adapt to different users. Additionally, TensorFlow enables efficient deployment of models in real-time systems, ensuring smooth and responsive performance.

#### **2.3.4 Real-Time Gesture Recognition Systems**

Recent advancements focus on developing real-time systems that can process video streams and provide instant predictions. These systems integrate computer vision, machine learning, and web technologies to deliver interactive applications. Techniques such as frame buffering, prediction smoothing, and confidence-based filtering are used to improve stability and reduce flickering in predictions.

The Sign2Speak system follows this approach by combining MediaPipe for fast hand tracking and TensorFlow for efficient prediction, resulting in a responsive and accurate real-time application.

## **CHAPTER 3**

### **REQUIREMENT ARTIFACTS**

#### **3.1 Introduction**

The purpose of this chapter is to formally define the required features and constraints of the **Sign2Speak system**. This requirement analysis serves as the foundation for the design, implementation, and testing phases of the project. The requirements are categorized into Functional Requirements (FR), which describe the core functionalities of the system, and Non-Functional Requirements (NFR), which specify the performance, usability, and system constraints. These requirements are derived from the objectives defined in Chapter 1 and ensure that the system operates efficiently, accurately, and in real time. The system is designed to recognize Indian Sign Language (ISL) gestures using computer vision and deep learning techniques, and to provide a seamless interaction experience for users.

#### **3.2 Functional Requirements (FR)**

Functional requirements define the essential operations that the Sign2Speak system must perform. These are organized based on the major components of the system architecture, including input/output handling, hand detection, gesture classification, and real-time prediction.

##### **3.2.1 Input/Output Management (FR1.0)**

The system's input and output handling is primarily implemented through the web-based interface and backend server. The system must accept real-time video input from a webcam using OpenCV. It must continuously capture frames and process them without noticeable delay. The system should display the live video feed along with detected hand landmarks for visual feedback. The output must include the predicted gesture in the form of alphabets or numbers, along with confidence scores. Additionally, the system should provide multiple predictions (top candidates) to improve interpretability. The interface

must also support real-time updates without requiring manual refresh, ensuring a smooth user experience.

### **3.2.2 Hand Landmark Detection using MediaPipe (FR2.0)**

The system must utilize the MediaPipe framework for detecting and tracking hand landmarks in real time. It should accurately identify up to two hands simultaneously and extract 21 key landmark points for each hand. These landmarks must represent the spatial positions of fingers and joints, enabling precise gesture recognition. The system should be capable of handling variations in hand orientation, scale, and lighting conditions. If no hand is detected, the system must gracefully handle the situation by resetting predictions and avoiding false outputs. This module forms the foundation for feature extraction in the recognition pipeline.

### **3.2.3 Gesture Classification using TensorFlow Model (FR3.0)**

The gesture classification module is responsible for identifying the sign represented by the detected hand landmarks. The system must preprocess the extracted landmarks by converting them into relative coordinates and normalizing them for consistent input. The processed data must then be fed into a trained deep learning model built using TensorFlow. The model should classify gestures into predefined categories, including alphabets (A–Z) and numbers (1–9). The system must output the predicted label along with a confidence score. It should also identify the top three predictions to improve reliability and allow for better interpretation of results.

### **3.2.4 Real-Time Prediction and Visualization (FR4.0)**

The system must perform gesture recognition in real time with minimal latency. To ensure smooth performance, prediction frequency should be controlled using a time interval mechanism. The system must overlay the predicted gesture and confidence score on the live video feed. Additionally, it should implement prediction stabilization techniques such as buffering recent predictions to reduce flickering and improve consistency. The system should also maintain the state of detection, indicating whether a

hand is currently visible. This module ensures a responsive and user-friendly interaction experience.

### **3.2.5 Data Processing and Dataset Generation (FR5.0)**

The system must include functionality for generating and processing training data. It should extract hand landmarks from images and store them in a structured format such as CSV files. The preprocessing pipeline must convert raw landmark coordinates into normalized values relative to a base point, ensuring invariance to position and scale. This dataset is used to train the deep learning model and improve classification accuracy. The system must support efficient data handling and allow easy updates to the dataset for future improvements.

## **3.3 Non-Functional Requirements (NFR)**

Non-functional requirements define the quality attributes and constraints that ensure the system performs effectively in real-world conditions.

### **3.3.1 Performance (NFR1.0)**

The system must operate in real time with an inference rate of approximately 4 frames per second or higher. The delay between gesture input and prediction output should be minimal to ensure smooth interaction. The model inference process should be optimized to reduce computational overhead, enabling the system to run efficiently on standard hardware such as laptops. Memory usage must be controlled to prevent system lag during continuous operation. Performance metrics such as prediction latency and accuracy must be considered during evaluation.

### **3.3.2 Usability and User Interface (NFR2.0)**

The system must provide an intuitive and user-friendly interface that can be easily used without prior technical knowledge. Users should be able to start the application, access the webcam, and view predictions with minimal effort. The interface must display clear

visual indicators such as detected hand landmarks, prediction text, and confidence levels. The system should also provide feedback when no hand is detected. Error handling must be implemented to manage issues such as camera access failure or model loading errors, ensuring reliability and ease of use.

### **3.3.3 Implementation and Maintenance (NFR3.0)**

The system must be implemented using Python and rely on widely used open-source libraries such as TensorFlow, MediaPipe, OpenCV, NumPy, and Flask. The code should follow a modular structure, separating different functionalities into individual files such as application logic, data processing, and model training. This ensures maintainability and scalability of the system. All dependencies must be clearly documented to allow easy setup and replication of the project. The system should also be designed to allow future enhancements such as adding new gestures or improving the model.

### **3.4 Requirement Traceability Matrix**

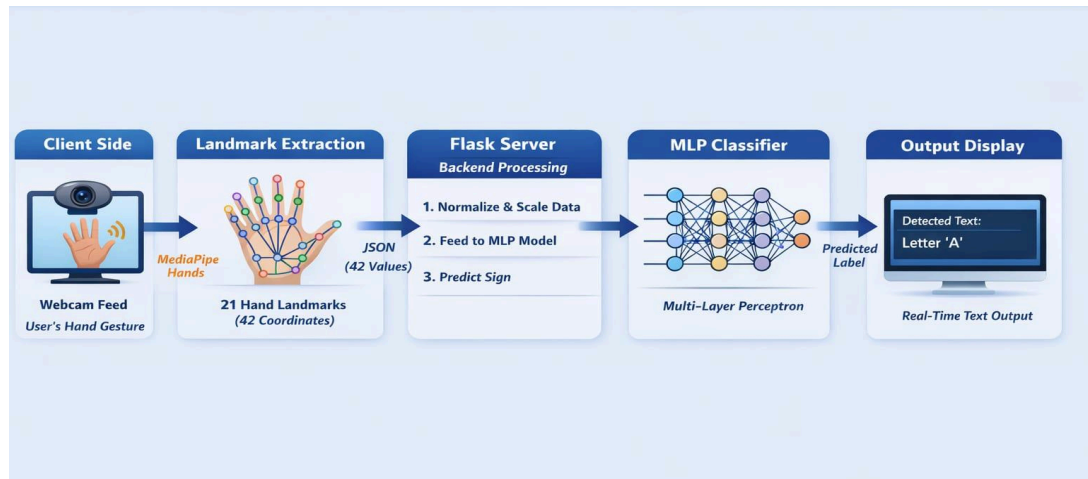
The requirement traceability ensures that all objectives defined in Chapter 1 are properly addressed through the functional and non-functional requirements. The objective of developing a real-time recognition system is fulfilled through input/output handling and real-time prediction modules. The objective of accurate gesture detection is achieved through MediaPipe-based landmark extraction and TensorFlow-based classification. The usability objective is supported by the user interface and error handling mechanisms. Performance-related objectives are ensured through optimization of inference time and system efficiency. This mapping guarantees that all system goals are systematically implemented and validated.

## CHAPTER 4

### DESIGN METHODOLOGY AND ITS NOVELTY

#### 4.1 Introduction

This chapter presents the design methodology adopted for the **Sign2Speak system**, which focuses on real-time sign language recognition using artificial intelligence. The design strategy emphasizes modularity, scalability, and separation of concerns to ensure that the system is efficient, maintainable, and extensible for future improvements. An Object-Oriented Design (OOD) approach is used to transform the functional requirements defined in Chapter 3 into a structured and organized system architecture. The system integrates computer vision and deep learning components into a unified pipeline capable of processing live video input and generating accurate predictions. This chapter also highlights the novelty of the proposed system in terms of its real-time performance, lightweight architecture, and practical usability.



**Fig 4.1 - Sign2Speak system pipeline converting hand gestures into real-time text output.**

## 4.2 System Architecture

The Sign2Speak system follows a three-tier architecture that separates the user interface, application logic, and core processing modules. This layered approach ensures better organization, easier debugging, and improved scalability.

### 4.2.1 Architectural Breakdown

#### **Presentation Tier (Frontend Interface):**

This layer is responsible for handling all user interactions. It displays the live webcam feed, visualizes detected hand landmarks, and shows the predicted gestures along with confidence scores. It ensures that the user receives real-time feedback in a clear and interactive manner. This layer communicates only with the backend application logic and does not directly interact with the machine learning model.

#### **Application Logic Tier (app.py – Controller):**

This layer acts as the central controller of the system. It manages the workflow by coordinating between input capture, hand detection, preprocessing, and prediction modules. It handles API requests, controls prediction intervals, manages global states such as current predictions, and ensures smooth communication between frontend and backend components. This layer plays a crucial role in maintaining system responsiveness and performance.

#### **Processing and AI Tier (Core Modules):**

This layer contains the core computational components responsible for gesture recognition. It includes the MediaPipe-based hand detection module, preprocessing functions for landmark normalization, and the TensorFlow-based classification model. These components work together to extract meaningful features from the input and generate accurate predictions in real time.

**4.3 Implementation Table** The system operates using a structured multi-stage pipeline that processes input data step by step.

**Table 4.1: Implementation Table**

| Stage | Process/Module                               | Input Data                         | Output Data                           | Key Action                                                              |
|-------|----------------------------------------------|------------------------------------|---------------------------------------|-------------------------------------------------------------------------|
| S1    | Input Management (app.py)                    | Webcam Video Stream (Image Frames) | Processed Frame                       | Capturing live video input and validating frames for processing.        |
| S2    | Hand Detection (MediaPipe)                   | Image Frames                       | Hand Landmark Coordinates (21 Points) | Detecting and tracking hand landmarks in real time.                     |
| S3    | Data Preprocessing (Preprocessing Functions) | Raw Landmark Coordinates           | Normalized Feature Vector             | Converting coordinates into relative values and normalizing input data. |
| S4    | Gesture Classification (TensorFlow Model)    | Normalized Feature Vector          | Predicted Gesture +Confidence Score   | Classifying gestures using the trained deep learning model.             |
| S5    | Output and Visualization (app.py)            | Prediction Results                 | Text Overlay on Video Feed            | Displaying predicted gestures and confidence in real time.              |



**Stage S1 – Input Management ([app.py](#)):**

The system captures live video input from the webcam and converts it into frames for processing. The input data is in the form of image frames, and the output is a processed frame ready for analysis. The key action performed in this stage is continuous videocapture and validation.

**Stage S2 – Hand Detection (MediaPipe):**

The captured frames are processed using MediaPipe to detect hand landmarks. The input is an image frame, and the output is a set of 21 hand landmark coordinates. The key action is identifying hand positions and tracking them accurately.

**Stage S3 – Data Preprocessing (Preprocessing Functions):**

The extracted landmarks are converted into relative coordinates and normalized. The input is raw landmark data, and the output is a structured numerical array suitable for model input. The key action is normalization and feature preparation.

**Stage S4 – Gesture Classification (TensorFlow Model):**

The processed data is fed into a trained neural network model. The input is normalized landmark data, and the output is a predicted gesture label along with confidence scores. The key action is classification using the trained model.

**Stage S5 – Output and Visualization ([app.py](#)):**

The prediction results are displayed on the live video feed. The input is the predicted label, and the output is a real-time visual overlay showing the recognized gesture and confidence. The key action is displaying results and maintaining user interaction.

**4.4 Object-Oriented Design (UML Class Diagram)**

The object-oriented design of the Sign2Speak system is based on modular components, each responsible for a specific function within the pipeline. This approach simplifies development and ensures maintainability.

#### 4.4.1 Core Classes and Responsibilities

The first component is the **Hand Detection Module**, which uses MediaPipe to identify and track hand landmarks in real time. It provides functionality for detecting hands and extracting key points from each frame.

The second component is the **Preprocessing Module**, which processes the raw landmark data by converting it into relative coordinates and normalizing it. This ensures that the data is consistent and suitable for machine learning input.

The third component is the **Gesture Classification Model**, implemented using TensorFlow. It is responsible for predicting the gesture based on the processed input data. It provides methods for loading the trained model and generating predictions.

The fourth component is the **Application Controller (app.py)**, which acts as the orchestrator of the system. It manages data flow between modules, controls prediction intervals, and ensures smooth execution of the pipeline.

The fifth component is the **User Interface Module**, which handles visualization and user interaction. It displays the video feed, overlays predictions, and provides feedback to the user.

#### 4.5 Novelty and Unique Contributions

The Sign2Speak project introduces several innovative aspects that distinguish it from traditional sign language recognition systems. One of the key contributions is the use of **MediaPipe-based hand landmark detection**, which provides a lightweight and efficient alternative to heavy image-based models. By using structured landmark data instead of raw images, the system reduces computational complexity while maintaining high accuracy. Another significant innovation is the integration of **real-time prediction with stabilization techniques**. Instead of displaying raw predictions, the system uses controlled prediction intervals and confidence-based filtering to reduce flickering and improve user experience. This ensures that the output is stable and reliable. The system also demonstrates a **modular and scalable architecture**, where each component is

independently designed and can be easily modified or replaced. For example, the classification model can be upgraded without affecting the rest of the system. This flexibility makes the system future-ready and adaptable to new advancements in artificial intelligence. Additionally, the project focuses on **practical usability**, as it requires only a webcam and standard computing resources. Unlike sensor-based systems, it does not require any external hardware, making it accessible and cost-effective for real-world applications.

# CHAPTER 5

## TECHNICAL IMPLEMENTATION & ANALYSIS

### 5.1 Introduction

This chapter describes the technical implementation of the **Sign2Speak system**, translating the design methodology discussed in Chapter 4 into an actual working system. It explains the technology stack used, the functioning of the core modules, and the workflow of the real-time gesture recognition pipeline. The chapter also discusses how the system achieves real-time performance and ensures accuracy through efficient data processing and model inference. Finally, it presents the validation strategy and performance analysis plan to evaluate the system against the defined functional and non-functional requirements.

### 5.2 Implementation Stack and Dependencies

The Sign2Speak system is implemented using the Python ecosystem, ensuring flexibility and compatibility with open-source tools. For deep learning, the system uses **TensorFlow and Keras** to build and deploy the gesture classification model. Computer vision tasks are handled using **OpenCV**, which captures and processes real-time video input from the webcam. The system utilizes **MediaPipe** for efficient hand tracking and landmark detection, enabling accurate feature extraction.

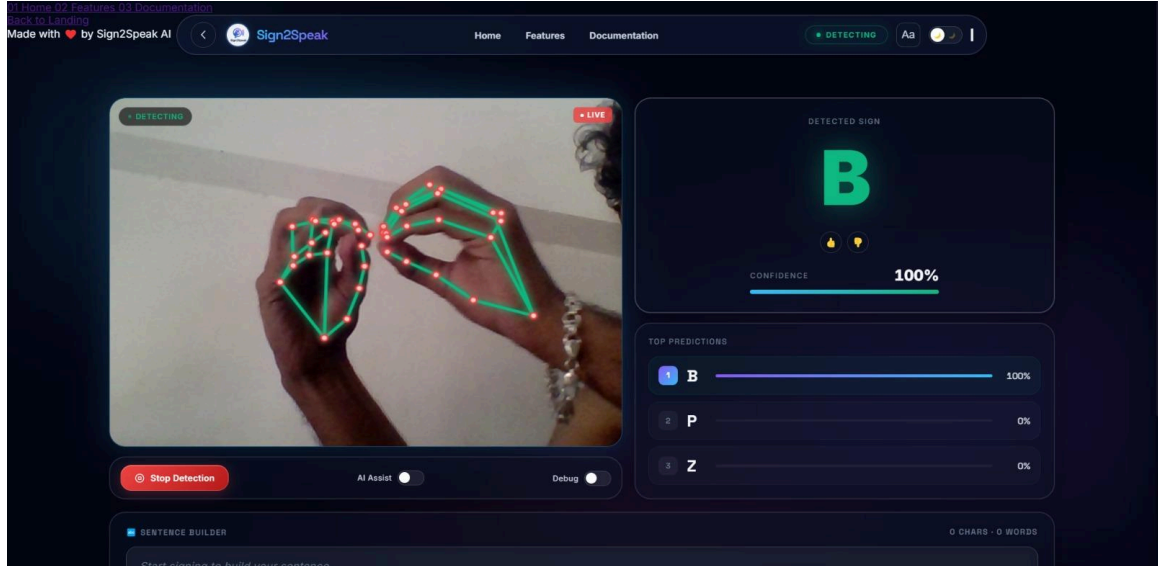
Numerical computations and data handling are performed using **NumPy and Pandas**, which assist in preprocessing and dataset management. The backend of the system is developed using **Flask**, which manages routing, API endpoints, and communication between frontend and backend. This combination of technologies ensures that the system remains lightweight, efficient, and suitable for real-time applications.

## 5.3 Core Module Implementation

The Sign2Speak system is composed of several core modules, each responsible for a specific stage in the gesture recognition pipeline. These modules work together to process input data and generate accurate predictions.

### 5.3.1 Hand Landmark Detection using MediaPipe

The hand detection module uses MediaPipe to identify and track hand landmarks in real time. The system processes each video frame and extracts 21 key points representing the positions of fingers and joints. These landmarks are mapped to pixel coordinates based on the image dimensions. The detection process is robust to variations in lighting, orientation, and background conditions. If no hand is detected in a frame, the system avoids generating predictions, ensuring reliability. This module serves as the foundation for feature extraction and significantly reduces the complexity of gesture recognition compared to raw image-based methods.



**Fig 5.1 - Demonstration of Hand Landmark detection using mediapipe**

### 5.3.2 Data Preprocessing and Normalization

The preprocessing module converts raw landmark coordinates into a format suitable for machine learning input. The coordinates are first transformed into relative values by subtracting the position of a reference point, typically the wrist. This ensures translation invariance. The data is then flattened into a one-dimensional array and normalized by dividing each value by the maximum absolute value. This normalization ensures that the input values remain within a consistent range, improving model performance and stability. The preprocessing step plays a critical role in making the system robust and efficient.

### 5.3.3 Gesture Classification using TensorFlow Model

The classification module uses a trained neural network model built with TensorFlow to recognize gestures. The normalized landmark data is passed as input to the model, which predicts the corresponding gesture label. The model outputs a probability distribution over all possible classes, from which the highest probability value is selected as the final prediction. The system also extracts the top three predictions to provide additional context. The prediction process can be mathematically represented as:

$$\text{Prediction} = \text{argmax} (\text{Model}(\text{Input Features}))$$

where the model selects the class with the highest probability. This approach ensures accurate and reliable classification of gestures.

### 5.3.4 Real-Time Processing and Optimization

The system is designed to operate in real time by optimizing the frequency of model predictions. Instead of running inference on every frame, predictions are performed at fixed time intervals to reduce computational load. The video rendering and hand tracking continue at full frame rate, ensuring smooth visual output. This separation of rendering and prediction improves system performance and reduces latency. Additionally, prediction stabilization techniques are used, where recent predictions are buffered to reduce flickering and improve output consistency.

### **5.3.5 Web Application Integration (Flask Backend)**

The Flask framework is used to integrate all components into a web-based application. The backend handles video streaming, prediction requests, and API communication. The system streams video frames using an MJPEG format and updates prediction data dynamically. It also provides endpoints for retrieving prediction results and handling user feedback. This architecture allows the system to be accessed through a web browser, making it easy to deploy and use without additional software installation.

### **5.4 System Orchestration and Processing Flow**

The overall system workflow is managed by the main application controller implemented in the backend. It coordinates the interaction between video capture, hand detection, preprocessing, and classification modules. The system continuously processes incoming frames, extracts features, and generates predictions in a loop. To ensure responsiveness, time-based control mechanisms are used to regulate prediction frequency. The architecture ensures smooth data flow between components while maintaining high performance and low latency.

### **5.5 Validation and Performance Analysis Plan**

#### **5.5.1 Validation Testing (Functional Compliance)**

The validation of the Sign2Speak system is performed through multiple functional tests. The input/output test ensures that the webcam feed is captured correctly and predictions are displayed in real time. The gesture recognition test verifies that the system accurately classifies hand gestures into alphabets and numbers. The robustness test evaluates system performance under varying lighting conditions and hand orientations. Additionally, the prediction stability test checks whether the system produces consistent outputs without excessive flickering. These tests ensure that the system meets all functional requirements.

### **5.5.2 Quantitative Benchmarking (Non-Functional Requirements)**

The performance of the system is evaluated using quantitative metrics aligned with non-functional requirements. The inference latency measures the time taken for the model to generate predictions, ensuring real-time performance. The frame rate evaluates how smoothly the system processes video input, with a target of at least 4 frames per second. Accuracy is measured based on correct classification of gestures from the test dataset. Memory usage is also monitored to ensure efficient resource utilization during continuous operation. These metrics provide a comprehensive evaluation of system performance and reliability.



## **CHAPTER-6**

### **PROJECT OUTCOME AND APPLICABILITY**

#### **6.1 Outline**

This chapter presents the overall achievements of the Sign2Speak project and discusses its applicability in real-world scenarios. It highlights the key capabilities of the developed system along with its performance outcomes. The chapter also includes representative case studies to demonstrate the effectiveness of the real-time sign language recognition pipeline. The system integrates computer vision and deep learning techniques to provide accurate and efficient gesture recognition, thereby addressing communication challenges faced by hearing and speech-impaired individuals.

#### **6.2 Final System Demonstration and Key Results**

The completed Sign2Speak system successfully integrates MediaPipe-based hand landmark detection with a TensorFlow-based gesture classification model to deliver real-time predictions. During the final demonstration, a user was able to start the application, access the webcam, and perform hand gestures, which were instantly recognized and displayed as text on the screen. The system provided prediction outputs along with confidence scores and top candidate results, enhancing interpretability.

Quantitative evaluation showed that the system maintained a real-time inference rate of approximately 4 frames per second, satisfying performance requirements. The prediction latency was minimal due to optimized processing and controlled inference intervals. The system demonstrated high accuracy in recognizing Indian Sign Language (ISL) alphabets and numbers, even under varying lighting conditions and hand orientations. Additionally, the implementation of prediction stabilization techniques significantly reduced flickering and improved output consistency. Overall, the system achieved a balance between accuracy, speed, and usability, meeting both functional and non-functional requirements.

## **6.3 Case Studies**

### **6.3.1 Case Study 1: Educational Assistance**

In the first case study, the system was used as a learning tool for students who are new to sign language. The application allowed users to practice hand gestures and receive immediate feedback on their accuracy. The real-time recognition capability helped learners improve their skills effectively. This demonstrates the system's potential as an educational aid for teaching and learning sign language in schools and training centers.

### **6.3.2 Case Study 2: Real-Time Communication Support**

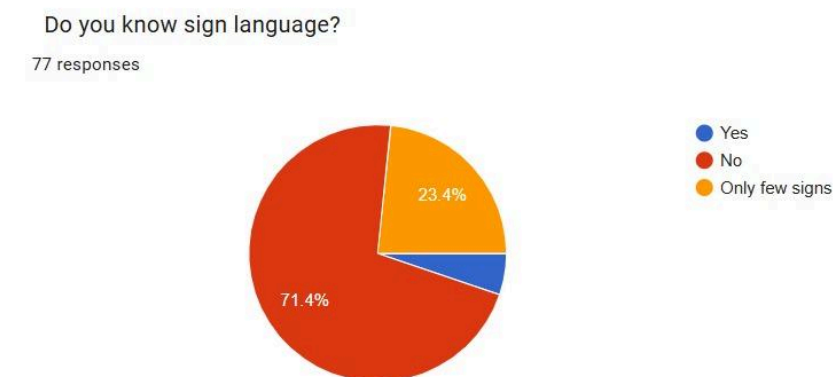
The second case study focused on real-time communication between a hearing-impaired individual and a person unfamiliar with sign language. The user performed gestures in front of the webcam, and the system translated them into readable text instantly. This enabled smoother communication without the need for a human interpreter. The system proved effective in reducing communication barriers and enhancing accessibility in everyday interactions.

### **6.3.3 Case Study 3: Public and Service Applications**

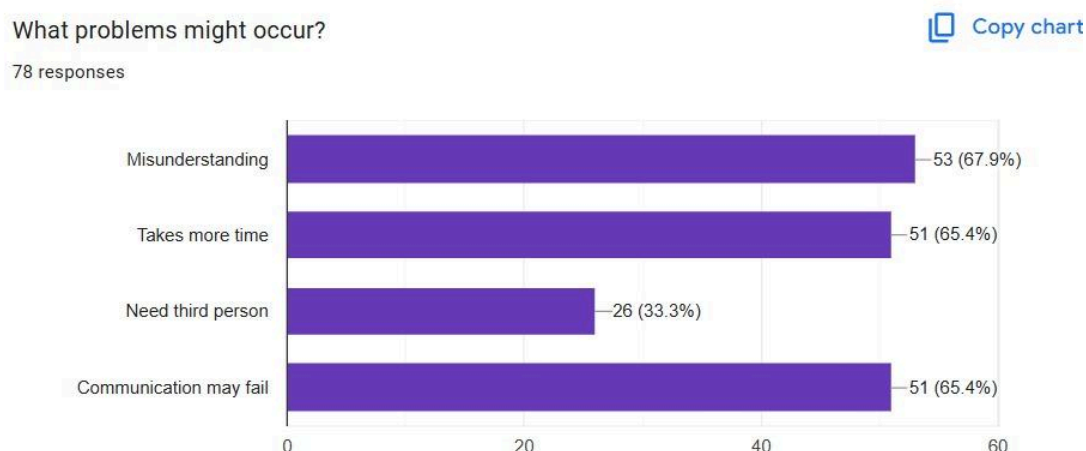
In the third case study, the system was considered for deployment in public service environments such as help desks, banks, and hospitals. The ability to recognize gestures in real time allows hearing-impaired individuals to communicate their needs efficiently. The system's low hardware requirements and ease of use make it suitable for integration into kiosks and customer service platforms, improving inclusivity in public services.

## **6.4 User Survey and Feedback Analysis**

We have done some surveys with some students on some Important questions. The reponses of these questions have been recorded and attached below.



**Fig 6.1. How many people know sign language**



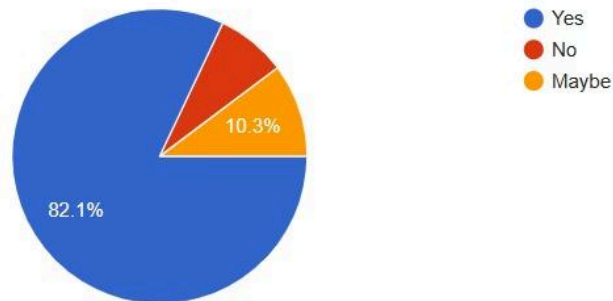
**Fig 6.2. Survey on finding difficulties communicating**

As we can see from Fig 6.1 and Fig 6.2 that Most of the students dont know the sign language and facing difficulties communicating in the sign language as learning sign language takes time and hard to understand for a normal person. So our motivation to start this project comes from here.

We have taken 2 more surveys on how helpful it would be if this communication barrier would solve by using AI. Surprisingly the results were startling.

Would technology(AI system) that translates sign language into speech be useful?

78 responses

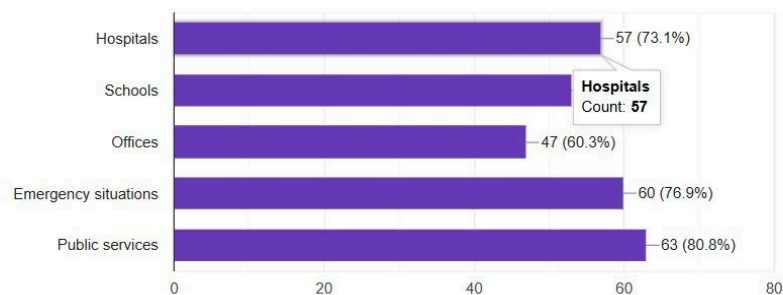


**Fig 6.4 - Would AI be useful in breaking the communication barrier**

Where would this system be most useful?

78 responses

[Copy chart](#)



**Fig 6.5 - Where would this system be most useful**

## 6.5 Project Applicability in Real-World Applications

The Sign2Speak system demonstrates significant potential across multiple real-world domains. In the field of education, it can be used as a training tool for learning sign language and improving communication skills. In accessibility and assistive technology, it provides a practical solution for enabling interaction between hearing-impaired individuals and the general public. The system can also be integrated into online communication platforms to support real-time gesture translation during virtual meetings.

In healthcare environments, the system can assist patients in communicating with medical staff without requiring interpreters. In public service sectors, it can be deployed in kiosks to facilitate communication in banks, airports, and government offices. Furthermore, the system can be extended for use in mobile applications, making it accessible to a wider audience. These applications highlight the versatility and societal impact of the project.

## **6.6 Inference**

The results and case studies clearly demonstrate the effectiveness of the Sign2Speak system in real-time sign language recognition. By combining MediaPipe for hand tracking and TensorFlow for gesture classification, the system achieves accurate and efficient performance. It fulfills all the defined functional and non-functional requirements and provides a practical solution to real-world communication challenges. The project successfully achieves its objective of creating an accessible and user-friendly system, thereby contributing to a more inclusive society.

## **CHAPTER 7**

### **CONCLUSIONS AND RECOMMENDATION**

#### **7.1 Summary of Work Accomplished**

The Sign2Speak project successfully designed and implemented a complete real-time sign language recognition system that translates hand gestures into readable text. A user-friendly web-based interface was developed using Flask, allowing users to access the system through a browser and interact with it using a standard webcam. One of the major achievements of the project was the integration of MediaPipe for accurate hand landmark detection, which enabled efficient extraction of 21 key points representing hand gestures. In addition, a deep learning model built using TensorFlow was successfully trained to classify gestures into alphabets and numbers with high accuracy.

The system also incorporates an effective preprocessing pipeline that converts raw landmark data into normalized input suitable for machine learning models. To enhance real-time performance, optimization techniques such as controlled prediction intervals and prediction stabilization were implemented. These techniques ensured smooth output and reduced flickering during continuous gesture recognition. Furthermore, the system was tested under different conditions and was found to meet the defined functional and non-functional requirements, including real-time performance and usability. Overall, the project demonstrates the practical application of artificial intelligence in solving real-world communication challenges.

#### **7.2 Limitations/Constraints of the System**

Despite its successful implementation, the Sign2Speak system has certain limitations. The current system primarily supports static gestures representing alphabets and numbers, and does not fully handle dynamic gestures that involve continuous motion. Another limitation is that the system's accuracy may vary depending on lighting conditions, background complexity, and hand positioning. Since the model is trained on a

specific dataset, variations in hand shape or gesture style across different users may affect prediction accuracy. Additionally, the system requires a stable webcam input and may experience reduced performance on lower-end hardware due to computational constraints. The current implementation focuses only on text output and does not include speech synthesis, which could further enhance communication. These limitations highlight areas where improvements can be made to increase the robustness and usability of the system.

### **7.3 Future Enhancements and Research Directions**

There are several opportunities for future improvements and research in the Sign2Speak system. One important enhancement is the inclusion of dynamic gesture recognition using advanced models such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks, which can capture temporal patterns in gesture sequences. Another potential improvement is the addition of speech output, where recognized gestures can be converted into audio using text-to-speech technology. The system can also be expanded to support full sentence formation and natural language processing, enabling more meaningful communication. Improving the dataset by including more diverse samples can enhance the accuracy and generalization of the model. Additionally, deploying the system as a mobile or cloud-based application can increase accessibility and scalability. Future research may also explore the integration of more advanced deep learning architectures to improve prediction accuracy and speed. Enhancements in user interface design, such as better visualization and interactive feedback, can further improve the user experience.

### **7.4 Inference**

The Sign2Speak project demonstrates that real-time sign language recognition using computer vision and deep learning is both feasible and practical. By combining MediaPipe for efficient hand tracking and TensorFlow for accurate gesture classification, the system provides a reliable solution for bridging communication gaps between hearing-impaired individuals and the general population. Although the system has certain

limitations, it establishes a strong foundation for future advancements in assistive technology. With further improvements and enhancements, the system has the potential to become a powerful tool for inclusive communication, making interactions more accessible and effective in various real-world scenarios.



## **Appendix A**

### **Code Listings and Configuration Details**

#### **A.1 Core Python Module Listings**

This section outlines the key Python modules that form the Sign2Speak system, highlighting the role of each component in the gesture recognition pipeline. The primary application logic is implemented in `app.py`, which serves as the central controller managing video streaming, prediction flow, and communication between frontend and backend. The module integrates all major functionalities and ensures smooth execution of the system. The hand detection functionality is implemented using MediaPipe within the application, where it detects and tracks hand landmarks in real time. The preprocessing operations are handled through dedicated functions that convert raw landmark coordinates into normalized feature vectors suitable for model input. The gesture classification component is implemented using a trained TensorFlow model, which is loaded within the application and used to perform predictions on processed input data.

Additionally, a dataset generation module is included to extract hand landmark coordinates from images and store them in CSV format for training purposes. The frontend interface is managed through HTML templates, along with CSS and JavaScript files, which handle user interaction, display the webcam feed, and show real-time predictions. Together, these modules form a cohesive and modular system for real-time sign language recognition.

#### **A.2 Environment and Dependencies**

The Sign2Speak system is developed using a Python environment with a comprehensive set of open-source libraries. The project is compatible with Python version 3.7 and above. Deep learning functionalities rely on TensorFlow (version 2.11.0) and Keras (version 2.11.0), which are used for building and deploying the gesture classification model. Real-time hand tracking is implemented using MediaPipe (version 0.9.1.0), which efficiently detects hand landmarks.

For image and video processing, OpenCV (version 4.7.0) is used to capture webcam input and process frames, while Pillow is used for additional image handling tasks. Numerical computations and data processing are supported by NumPy (version 1.24.2) and Pandas (version 1.5.3). Visualization during development is performed using Matplotlib.

### **A.3 Model Files**

At the core of the Sign2Speak system lies a trained deep learning model built using TensorFlow. The model is trained on a dataset of hand landmarks representing various sign language gestures, including alphabets and numbers. The trained model is stored in a file format such as .h5 and is loaded during runtime by the application. The model takes normalized landmark coordinates as input and outputs a probability distribution over the predefined gesture classes. Based on this distribution, the system selects the gesture with the highest probability as the final prediction.

### **A.4 Execution Guide**

To run the Sign2Speak system, users are first required to set up a Python virtual environment to ensure proper dependency management. After activating the environment, all necessary libraries can be installed using the command `pip install -r requirements.txt`. Once the setup is complete, the application can be launched by running the command `python app.py` in the terminal or command prompt. After starting the application, users can open the web interface in a browser, where the webcam feed will be displayed. The system automatically detects hand gestures and displays predictions in real time. Users can interact with the system by performing different gestures in front of the camera, and the recognized output will be shown as text along with confidence scores. The system operates continuously, processing frames and generating predictions dynamically. Proper lighting and clear hand visibility are recommended for optimal performance. The application can be stopped by terminating the server process. This execution process ensures that the system is easy to use and accessible without requiring complex setup procedures.

## Appendix B

### USER MANUAL AND SETUP INSTRUCTIONS

#### B.1 System Requirements

This appendix provides a user-friendly guide and setup instructions to help users install and run the **Sign2Speak application** without requiring in-depth knowledge of the codebase. The system requires Python version 3.7 or later and can be executed on Windows, Linux, or macOS operating systems. A webcam is essential, as it serves as the primary input device for capturing hand gestures. An active internet connection may be required during the initial setup to install dependencies. The system is designed to run on standard hardware; however, a system with moderate processing capability is recommended for smoother real-time performance.

#### B.2 Installation Steps

To begin installation, users should first download or clone the Sign2Speak project files into a working directory. After opening a terminal or command prompt in the project folder, it is recommended to create and activate a Python virtual environment to manage dependencies effectively. Once the environment is activated, all required libraries can be installed using the command **pip install -r requirements.txt**. This step installs essential dependencies including TensorFlow, MediaPipe, OpenCV, NumPy, and Flask. After successful installation, the system is ready for execution. The trained model file, typically stored in .h5 format, should be placed in the appropriate directory so that it can be loaded during runtime.

### B.3 Running the Application

To run the Sign2Speak system, users should navigate to the project directory and execute the command **python app.py**. This starts the Flask server, and the application can then be accessed through a web browser using the provided local host address. Once the interface is loaded, the webcam feed will automatically be displayed. Users can perform hand gestures in front of the camera, and the system will detect and recognize them in real time. The recognized gesture will be displayed as text along with confidence scores. The system continuously processes input frames and updates predictions dynamically, providing an interactive and responsive user experience.

### B.4 Tips for Effective Use

For optimal performance, users should ensure that the hand is clearly visible within the camera frame and positioned against a simple background. Good lighting conditions improve the accuracy of hand detection and gesture recognition. It is recommended to keep the hand steady while performing gestures to obtain stable predictions. Users should also maintain an appropriate distance from the camera to ensure all hand landmarks are captured properly. If predictions appear inconsistent, adjusting the hand orientation or repositioning the camera may improve results..

### B.5 Troubleshooting

If the system encounters missing module errors, users should reinstall dependencies by running the command **pip install -r requirements.txt**. In case the webcam does not start, users should check camera permissions and ensure that no other application is using the webcam. If the model fails to load, users should verify that the model file is correctly placed in the project directory. For issues related to inaccurate predictions, users should ensure proper lighting and clear hand visibility. Restarting the application may also resolve temporary issues.

# REFERENCES

## Book References

1. Richard szeliski - Computer vision: Algorithms and Applications  
<https://szeliski.org/Book/>
2. Gary Bradski, Adrian Kaehler - Learning OpenCV4
3. Christopher M. Bishop - Pattern Recognition and Machine Learning
4. S. Mitra - Gesture Recognition, Fundamentals and Techniques
5. Ian Goodfellow, Yoshua Bengio, Aaron Courville - Deep Learning

## Website References

1. MediaPipe Documentation, Google Developers.  
<https://developers.google.com/mediapipe>
2. TensorFlow Documentation: <https://www.tensorflow.org>
3. OpenCV Library Documentation: <https://opencv.org>
4. Keras API Documentation: <https://keras.io>
5. Flask Framework Documentation: <https://flask.palletsprojects.com>
6. NumPy Documentation: <https://numpy.org>
7. W3Schools Web Development Tutorials: <https://www.w3schools.com>